Data Warehousing & Business Intelligence

7BUIS010W

Assessment 2

Contributors

Michael Ewer – W1778362 (Task A)

Razin Haque – W2125537 (Task B)

Stephen Ugbana – W2187654 (Task C)

For Notebooks & Figures Refer To GitHub Link Below

GitHub Repo: https://github.com/Raz-Haque/sp500-portfolio-analysis

Table of Contents

Task A ***4***

Figure 1. Tickers List 4

Figure 2. Dataset Shape 4

Figure 3. Cleaning (Before & After) 4

Figure 4. Daily Returns 5

Figure 5. Beta Table 5

Figure 6. Volatility Table 6

Figure 7. Average Daily Returns Table 6

Figure 8. Summary Table 7

Figure 9. Graph 1 (Beta Vs Return) 7

Figure 10. Graph 2 (Volatility Distribution) 8

Process for Data Collection and Analysis 8

Data Collection 8

Data Cleaning 9

Return Calculation 9

Beta Calculation 9

Volatility Calculation 9

Summary Metrics 9

Visual Analysis 10

Task B ***11***

Agglomerative Clustering 11

Appropriateness of Agglomerative clustering 11

Agglomerative Clustering Implementation 11

Figure 11. Silhouette plot for 2-9 clusters 12

Figure 12. Plot for k=4 cluster segmentation. 13

K-means Clustering 14

Appropriateness of K-means clustering 14

Remove Skewness and Scaling 14

Figure 13. Histogram of distribution of input data before normalisation. 15

Figure 14. Distribution of input data prior to normalisation. 16

K-means Clustering Implementation 17

Figure 16. K=4 cluster segmentation plot. 18

Figure 17. K=4 cluster segmentation plot. 19

Task C 20

C-1a. Identification of business value customer segments/location 20

C-1b. Justification of their business value 20

C-2a. Review of Results 20

C-2b. Python Visualisations 21

C-3. How CRM tools contribute to sustainability for Fintech 23

Task A

Figure 1. Tickers List

The output below shows the S&P 500 companies successfully extracted from Wikipedia, providing the list of stock tickers used in the analysis.

```python
import pandas as pd
import requests

url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"

headers = {
    "User-Agent": "Mozilla/5.0"
}

response = requests.get(url, headers=headers)

tables = pd.read_html(response.text)
sp500_table = tables[0]

tickers = sp500_table["Symbol"].tolist()

tickers = [ticker.replace(".", "-") for ticker in tickers]

print("Number of tickers:", len(tickers))
print(tickers[:10])
```

```
/tmp/ipykernel_385/1117526609.py:12: FutureWarning: Passing literal html to 'read_html' is deprecated and will be removed in a future version. To read from a literal string, wrap it in a 'StringIO' object.
  tables = pd.read_html(response.text)
Number of tickers: 503
['MMM', 'AOS', 'ABT', 'ABBV', 'ACN', 'ADBE', 'AMD', 'AES', 'AFL', 'A']
```

Figure 2. Dataset Shape

This confirms that price data was successfully downloaded for all tickers over the selected time.

```python
full_price_data = yf.download(tickers, start=start_date, end=end_date, auto_adjust=False)["Adj Close"]

print("Full dataset shape:", full_price_data.shape)
full_price_data.head()
```

```
[*********************100%***********************]  503 of 503 completed
ERROR:yfinance:
2 Failed downloads:
ERROR:yfinance:['SNDK', 'Q']: YFPricesMissingError('possibly delisted; no price data found  (1d 2022-01-01 -> 2025-01-01) (Yahoo error = "Data doesn\'t exist for startDate = 1641013200, endDate = 1735707600")')
Full dataset shape: (753, 503)
```

Ticker	A	AAPL	ABBV	ABNB	ABT	ACGL	ACN	ADBE	ADI	ADM	...	WY	WYNN	XEL	XOM	XYL	XYZ	YUM	ZBH	ZBRA	ZTS
Date																					
2022-01-03	151.702148	178.103653	115.847214	172.679993	128.345963	42.362530	383.711945	564.369995	164.526443	60.037254	...	33.979385	85.125763	59.164780	54.760239	110.957497	164.029999	125.887230	120.797722	583.900024	224.021271
2022-01-04	146.573669	175.843262	115.624794	170.800003	125.327499	42.914051	380.969879	554.000000	163.040482	61.151539	...	34.414906	84.854210	59.539249	56.820000	112.134560	156.330002	126.560310	122.023201	587.599976	215.492294
2022-01-05	144.062714	171.165863	116.232170	162.250000	124.764427	42.410072	374.260742	514.429993	160.523575	60.673988	...	33.560608	81.711830	60.122723	57.526684	110.606270	143.490005	124.955948	121.620941	558.179993	207.298370
2022-01-06	144.566849	168.308502	115.684662	159.750000	124.745941	42.657307	356.187469	514.119995	161.034409	61.213432	...	33.828621	82.090057	59.652454	58.879745	110.454376	144.660004	126.246811	120.610626	555.159973	208.121597
2022-01-07	140.718079	168.474884	115.385269	166.050003	125.133614	42.856995	349.355835	510.700012	156.808609	61.735191	...	33.786743	82.002777	60.174950	59.362370	109.571594	141.539993	124.799187	119.880959	530.859985	202.062286

5 rows × 503 columns

Figure 3. Cleaning (Before & After)

This step removes companies with missing data to ensure consistency and accuracy in subsequent calculations.

```python
print("Before cleaning:", full_price_data.shape)
clean_data = full_price_data.dropna(axis=1)
print("After cleaning:", clean_data.shape)
```

```
Before cleaning: (753, 503)
After cleaning: (753, 495)
```

Figure 4. Daily Returns

Daily returns are calculated to measure the percentage change in stock prices, forming the basis for risk and performance analysis.

```
daily_returns = clean_data.pct_change().dropna()
daily_returns.head()
```

Ticker	A	AAPL	ABBV	ABNB	ABT	ACGL	ACN	ADBE	ADI	ADM	...	WY	WYNN	XEL	XOM	XYL	XYZ	YUM	ZBH	ZBRA	ZTS
Date																					
2022-01-04	-0.033806	-0.012691	-0.001920	-0.010887	-0.023518	0.013019	-0.007146	-0.018374	-0.009032	0.018560	...	0.012817	-0.003190	0.006329	0.037614	0.010608	-0.046943	0.005347	0.010145	0.006337	-0.038072
2022-01-05	-0.017131	-0.026600	0.005253	-0.050059	-0.004493	-0.011744	-0.017611	-0.071426	-0.015437	-0.007809	...	-0.024823	-0.037033	0.009800	0.012437	-0.013629	-0.082134	-0.012677	-0.003297	-0.050068	-0.038024
2022-01-06	0.003499	-0.016694	-0.004710	-0.015408	-0.000148	0.005830	-0.048291	-0.000603	0.003182	0.008891	...	0.007986	0.004629	-0.007822	0.023521	-0.001373	0.008154	0.010331	-0.008307	-0.005410	0.003971
2022-01-07	-0.026623	0.000989	-0.002588	0.039437	0.003108	0.004681	-0.019180	-0.006652	-0.026242	0.008524	...	-0.001238	-0.001063	0.008759	0.008197	-0.007992	-0.021568	-0.011467	-0.006050	-0.043771	-0.029114
2022-01-10	0.000069	0.000116	0.011195	-0.032159	-0.002213	0.021078	0.006069	0.029626	0.009240	0.001719	...	-0.004709	-0.028149	0.010999	-0.005953	-0.010136	0.020772	-0.013964	-0.009754	0.008571	0.007154

5 rows x 495 columns

Figure 5. Beta Table

Beta values measure each stock's sensitivity to market movements, indicating relative risk compared to the overall market.

```
beta_values = {}

for ticker in daily_returns.columns:
    combined = pd.concat([daily_returns[ticker], market_returns], axis=1).dropna()
    combined.columns = ["stock", "market"]

    corr = combined["stock"].corr(combined["market"])
    beta = corr * (combined["stock"].std() / combined["market"].std())

    beta_values[ticker] = beta

beta_df = pd.DataFrame(list(beta_values.items()), columns=["Ticker", "Beta"])
beta_df.head()
```

	Ticker	Beta
0	A	1.024800
1	AAPL	1.210690
2	ABBV	0.299098
3	ABNB	1.666256
4	ABT	0.672317

Figure 6. Volatility Table

Annual volatility represents the level of risk associated with each stock, calculated from the standard deviation of returns.

```
volatility = daily_returns.std() * np.sqrt(252)

volatility_df = pd.DataFrame({
    "Ticker": volatility.index,
    "Annual Volatility": volatility.values
})

volatility_df.head()
```

	Ticker	Annual Volatility
0	A	0.296259
1	AAPL	0.270935
2	ABBV	0.219958
3	ABNB	0.467875
4	ABT	0.217839

Figure 7. Average Daily Returns Table

This shows the average return generated by each stock, providing a measure of overall performance.

```
avg_return = daily_returns.mean()

returns_df = pd.DataFrame({
    "Ticker": avg_return.index,
    "Average Daily Return": avg_return.values
})

returns_df.head()
```

	Ticker	Average Daily Return
0	A	-0.000002
1	AAPL	0.000591
2	ABBV	0.000610
3	ABNB	0.000074
4	ABT	-0.000106

Figure 8. Summary Table

This shows the average return generated by each stock, providing a measure of overall performance.

```
summary_df = returns_df.merge(beta_df, on="Ticker").merge(volatility_df, on="Ticker")
summary_df.head()
```

	Ticker	Average Daily Return	Beta	Annual Volatility
0	A	-0.000002	1.024800	0.296259
1	AAPL	0.000591	1.210690	0.270935
2	ABBV	0.000610	0.299098	0.219958
3	ABNB	0.000074	1.666256	0.467875
4	ABT	-0.000106	0.672317	0.217839

Figure 9. Graph 1 (Beta Vs Return)

This scatter plot illustrates the relationship between risk (beta) and return, helping identify whether higher risk leads to higher returns.

```
plt.figure(figsize=(8,6))
plt.scatter(summary_df["Beta"], summary_df["Average Daily Return"])
plt.xlabel("Beta")
plt.ylabel("Average Daily Return")
plt.title("Beta vs Average Daily Return")
plt.grid(True)
plt.show()
```

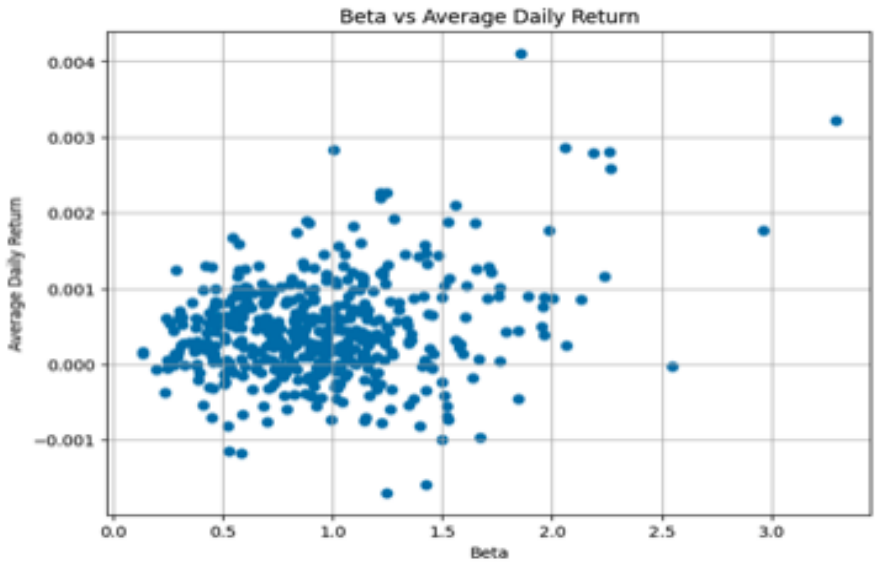
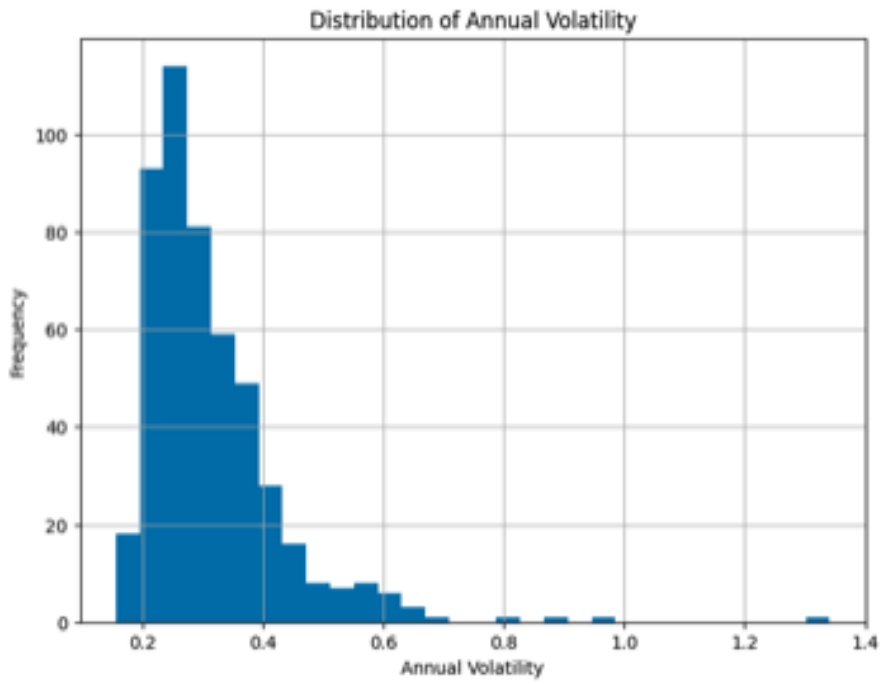


Figure 10. Graph 2 (Volatility Distribution)

```
plt.figure(figsize=(8,6))
plt.hist(summary_df["Annual Volatility"], bins=30)
plt.xlabel("Annual Volatility")
plt.ylabel("Frequency")
plt.title("Distribution of Annual Volatility")
plt.grid(True)
plt.show()
```



This histogram shows the distribution of stock volatility, highlighting how risk levels vary across the S&P 500.

Process for Data Collection and Analysis

Data Collection

Stock data for S&P 500 companies was collected using Python. The list of companies was extracted from Wikipedia and processed to obtain valid stock ticker symbols. Historical price data was then downloaded using the finance library for the period January 2022 to January 2025.

Data Cleaning

The dataset initially contained missing values for some companies. To ensure consistency, any stocks with incomplete data were removed. This resulted in a clean dataset with fewer tickers but complete observations across the entire time period.

Return Calculation

Daily returns were calculated using percentage changes in adjusted closing prices. These returns form the foundation for analysing stock performance and risk.

Beta Calculation

Beta was calculated for each stock to measure its sensitivity to overall market movements. A higher beta indicates greater volatility relative to the market, while a lower beta suggests more stable performance.

Volatility Calculation

Volatility was calculated as the standard deviation of daily returns and annualised using the square root of 252 trading days. This provides a measure of the risk associated with each stock.

Summary Metrics

A summary dataset was created by combining average returns, beta values, and volatility. This allows for direct comparison of stocks based on both performance and risk.

Visual Analysis

Two visualisations were created to support the analysis and provide insight into how risk and return vary across different stocks:

- A scatter plot of beta versus average daily return shows the relationship between risk and return.
- A histogram of annual volatility illustrates the distribution of risk across the S&P 500.

Task B

Agglomerative Clustering

Appropriateness of Agglomerative clustering

Agglomerative clustering is an appropriate clustering technique for this analysis as it is well suited to exploratory financial segmentation where it is not clear to the natural structure of the data. Agglomerative clustering builds clusters hierarchically by progressively merging similar observations. This makes it an effective modelling technique to identify the nested relationships between stocks and their varying risk profiles.

For the clustering process, the summary features engineered during the previous task would be the features used for modelling the clusters. Average daily return, annual volatility and beta are all continuous variables that observe gradual transitions in stocks rather than sharply separated groups. This makes agglomerative clustering particularly suitable in this context as it does not assume spherical cluster shapes and can reveal meaningful and insightful hierarchal groupings among the stocks with similar market behaviour. Furthermore, the interpretability of agglomerative clustering makes it effective at producing actionable portfolio grouping.

However, a potential limitation of this method is the computational power necessary can become expensive and it can also be distorted by extreme outliers. Despite this, its overall ability to uncover structured stock relationships affirms it as a highly appropriate method for this stock portfolio analysis.

Agglomerative Clustering Implementation

The optimal number of clusters was investigated using the silhouette method. The silhouette score was calculated and put into a plot for 2-9 clusters.

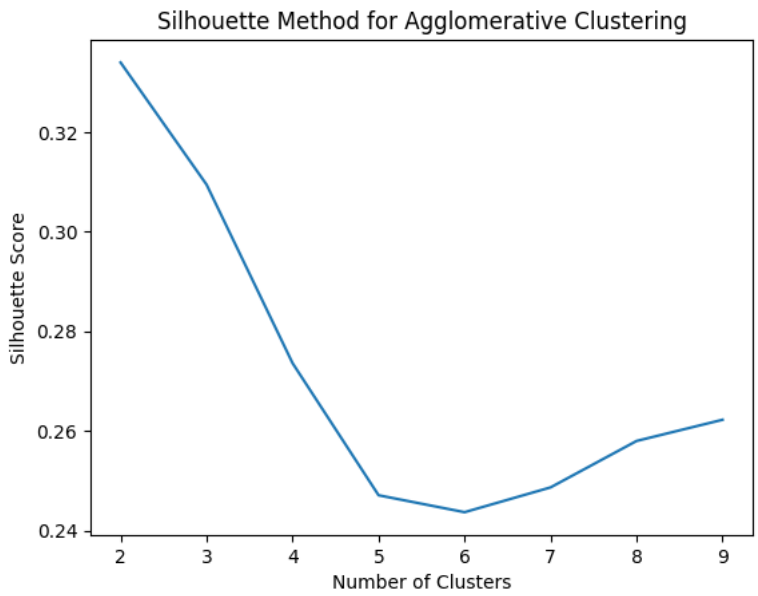


Figure 11. Silhouette plot for 2-9 clusters

3,4 and 5 clusters were moved forward for further visual analysis as they showed strong silhouette scores while also offering a richer and more informative segmentation structure.

In order to further investigate the cluster segmentation of these clusters, each number of clusters was put into an agglomerative clustering model. A plot for each model was then made for visual analysis.

After visually comparing the three cluster model solutions, the four-cluster model was selected as the most appropriate configuration because it provided the strongest balance between interpretability, granularity, and meaningful financial segmentation.

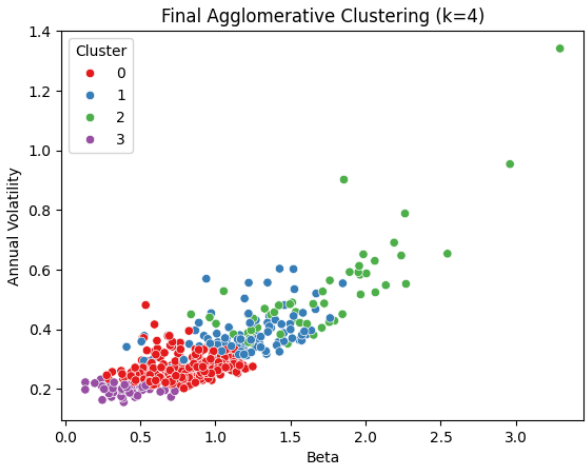


Figure 12. Plot for k=4 cluster segmentation.

While the three-cluster solution presented clear separation among cluster groups, it grouped too many medium-risk stocks into broad middle categories. This would limit the level insight that could be drawn from any analysis. This would also mean that values with largely different beta and volatility value characteristics were merged in the same clusters, this therefore meant no distinction between lower-medium risk and higher-medium risk could be made.

The four-cluster solution would add an additional level of segmentation in this regard, without overcomplicating the model. This meant that clusters could be categorized into more meaningful and insightful subgroups, especially distinguishing between defensive low-risk stocks, stable core holdings, moderate growth stocks, and aggressive high-risk stocks. In terms of portfolio investment, this model would be more aligned to reflecting different investment strategies.

While the five cluster model would potentially offer greater detail in terms of sub-group, it was not chosen as the optimal model as it began to over-segment the lower risk region of the data, Several clusters in the low-beta, low-volatility range became fragmented into groups that were visually close together and financially similar, making them harder to justify as genuinely distinct investment categories. Therefore, continuing with this model would risk diluting the usefulness for portfolio analysis.

K-means Clustering

Appropriateness of K-means clustering

K-means clustering is highly effective for partitioning numerical financial data into interpretable groups based on similarity within groups. The algorithm is designed to cluster values by minimising the distance between data points and their assigned cluster centroid. In terms of stock-risk analysis, this allows similar risk-return characteristics to be grouped together efficiently, making it suited for portfolio segmentation.

The simplicity and in turn lack of computational power needed to run, makes it potentially more suited for financial analysis. It is much faster to compute than agglomerative clustering on large datasets and produces clear centroid-based clusters that are fairly easy to interpret.

Remove Skewness and Scaling

Before proceeding with the clustering, it was important to check the input data for skewness. This is because k-means is highly sensitive to feature distributions and scale. K-means assigns clusters by calculating Euclidean distance of points based around a cluster centroid. However, if there are large numerical ranges or data that is highly skewed, it can dominate distance calculations and therefore distort the results.

In order to identify this visually, the input data for the 3 features were plotted in a histogram to identify the shape of the distribution.

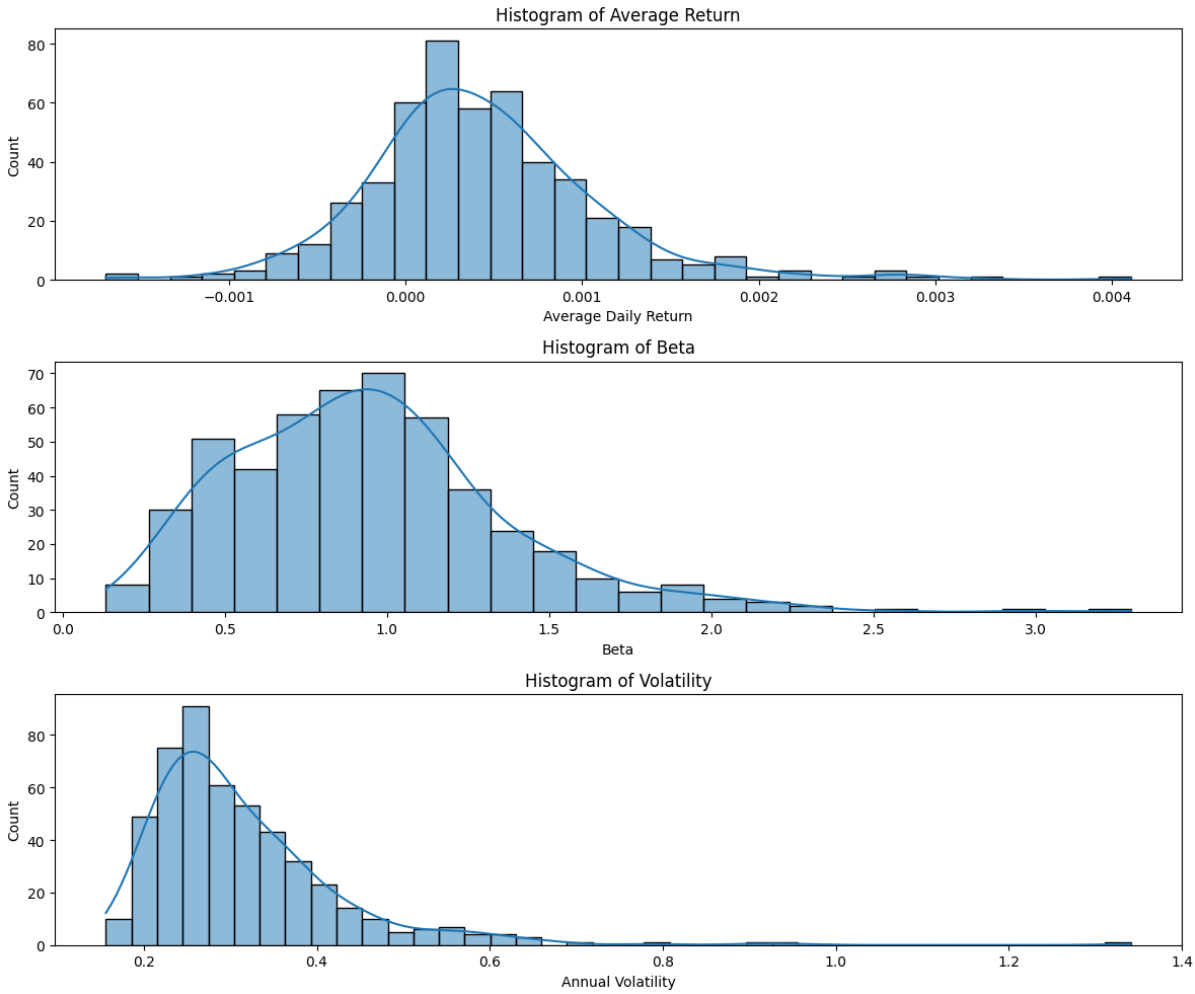


Figure 13. Histogram of distribution of input data before normalisation.

The plots showed that all 3 of the variable distributions displayed skewness. Beta and annual volatility in particular showed heavy left-sides skewness.

To remove this skewness, the data would need to be transformed, the chosen method of transformation was Box-Cox transformation. Although, average daily return could not be directly put into this algorithm as it contained negative values and in order to perform any logarithm, the values would need to all be positive in order to the output to be valid.

To address this, average daily return was shifted upward so that all values would become positive. All the input values were then put through a box-cox transformation.

The distributions for each feature were then replotted for visual skew analysis.

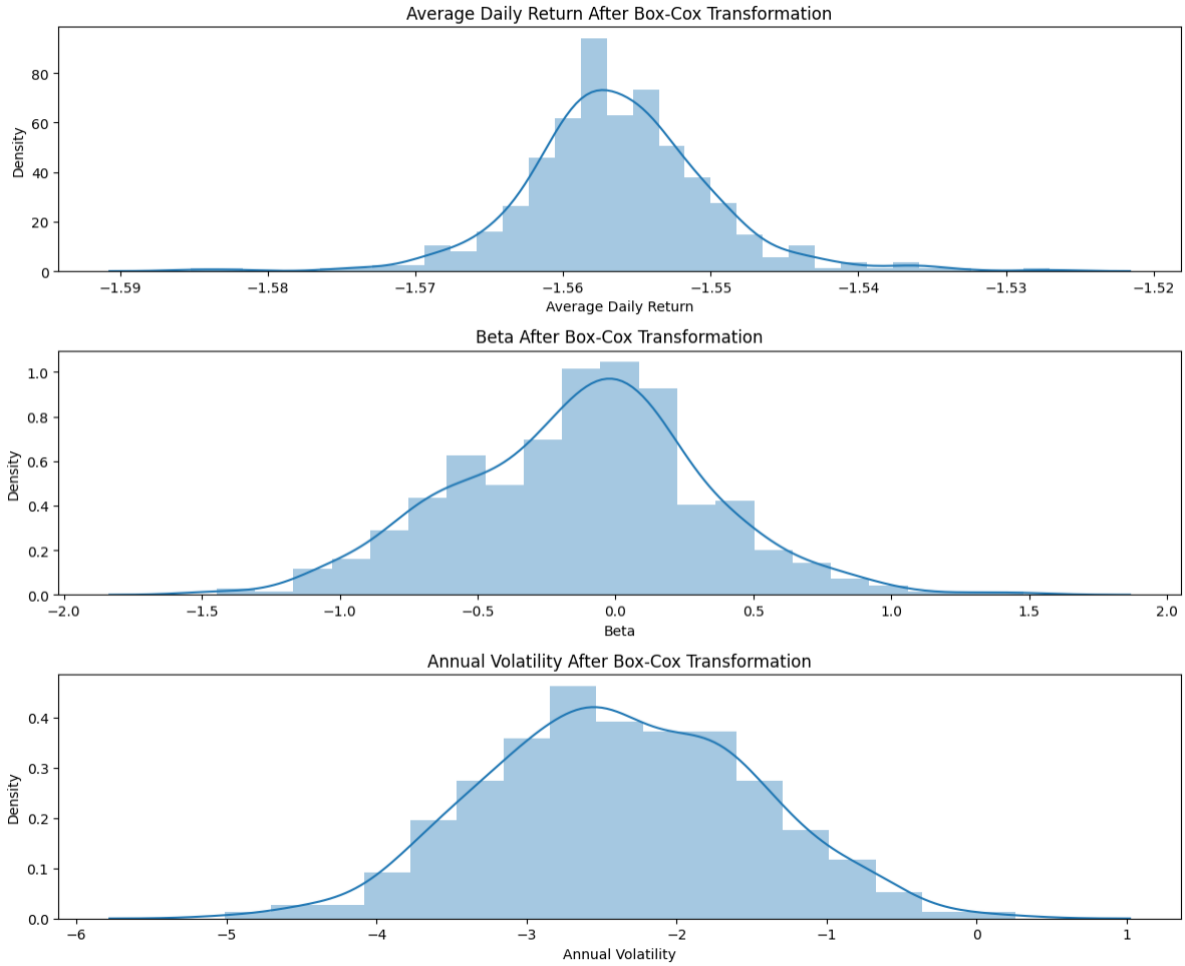


Figure 14. Distribution of input data prior to normalisation.

The new plots show a much more even and symmetrical distribution and confirms normalisation of the input data.

After the removal of skewness in the data, all the transformed variables were standardised using the StandardScaler. This would ensure equal contribution in distance-based clustering.

Scaling is also vitally important as the 3 variables operate on different numerical scales; therefore, it needs to be standardised as variables that inherently contain larger magnitudes will dominate the distance calculation. To ensure each variable would equally contribute to centroid formation the variables were all scaled.

K-means Clustering Implementation

In order to determine the optimal number of clusters, initially the elbow method was used. This technique evaluates cluster performance by calculating inertia between data points and their centroids. The inertia for cluster values from 1 to 10 were generated and plotted using the matplotlib library.

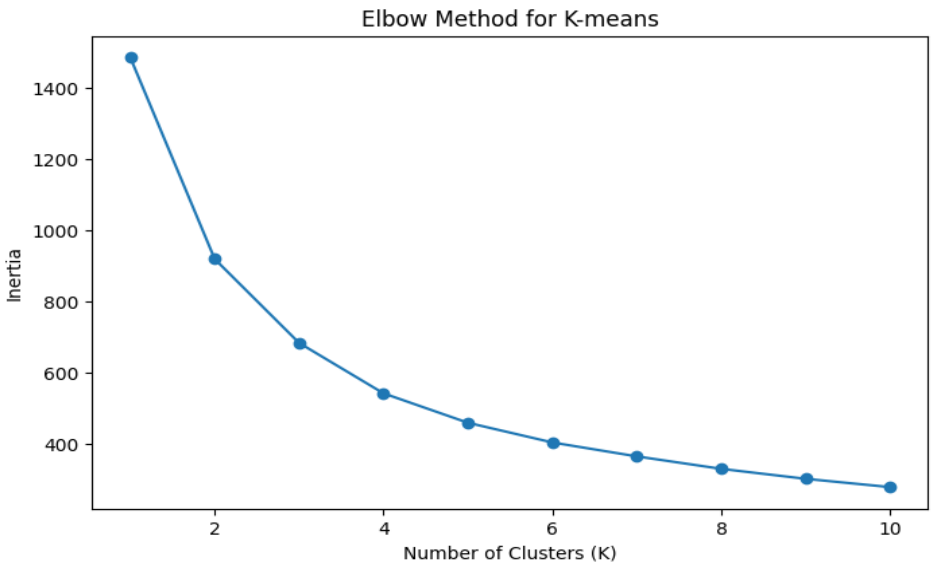


Figure 15. Graph of inertia values for k-means models from 1-10 cluster values.

The results showed a sharp decrease in inertia values from 1 to 3 clusters; this was then followed by a noticeable reduction in improvement after 4 clusters. This is represented as the elbow and represents the point of diminishing returns. Based off this cluster values around 3 and 4 were identified as the strongest candidates.

Given the results from the elbow method, cluster values of 3 and 4 were evaluated further through visual inspection.

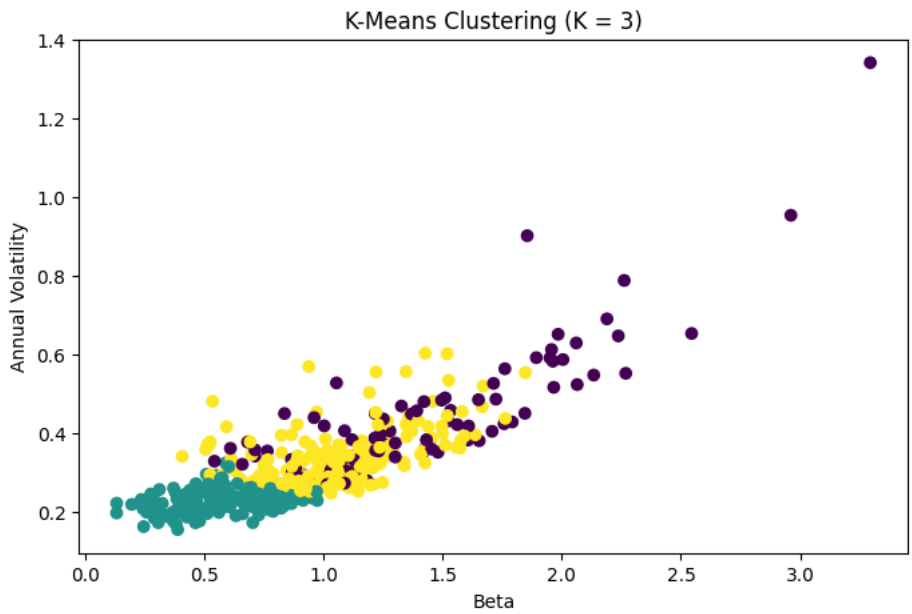


Figure 16. K=4 cluster segmentation plot.

The k=3 solution produces three clearly defined groups that correspond loosely to low-risk, medium-risk and high-risk stocks. Although, despite the clear separation between groups, the middle cluster in particular lacks any real depth as it is extremely broad. This therefore limits its relevance to a detailed portfolio analysis.

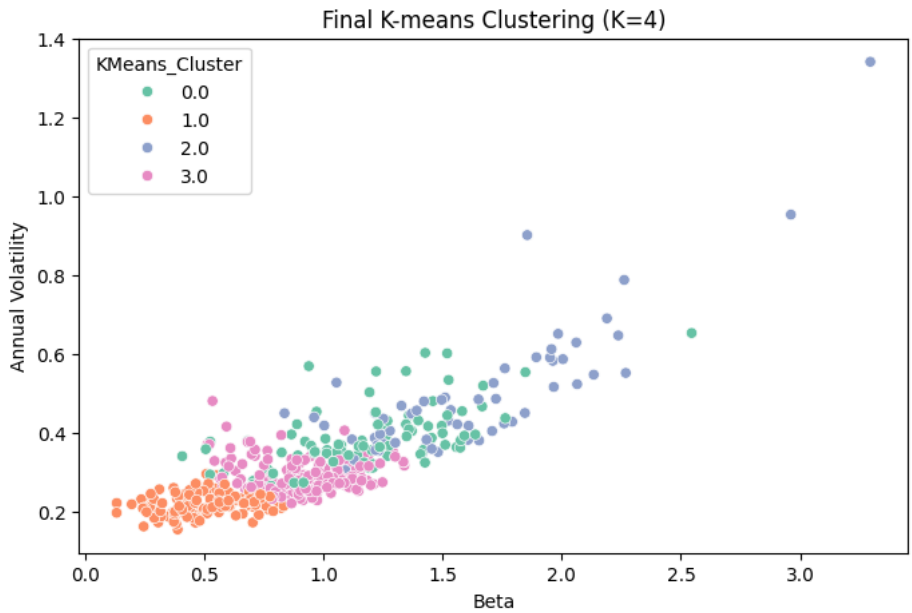


Figure 17. K=4 cluster segmentation plot.

The k=4 solution provides a more refined segmentation of the data. This extra level of detail is valuable for portfolio analysis and allows four interpretable and financially meaningful categories.

- Defensive low-risk stocks (low beta, low volatility)
- Stable core holdings (moderate beta, low volatility)
- Moderate growth stocks (higher beta, moderate volatility)
- Aggressive high-risk stocks (high beta, high volatility)

As a result, K=4 was selected as the optimal number of clusters for the final k-means model.

Task C

C-1a. Identification of business value customer segments/location

Based on Task B, the data mart should centre on a Fact_Stock_Performance table at the grain of one stock per trading day, supported by DimDate, DimStock, DimBenchmark, and DimRiskSegment. The core metrics stored in the fact table should be Daily Return, Beta, and Annual Volatility, because these were the three variables used in Python to segment the S&P 500 stocks and evaluate their investment behaviour.

C-1b. Justification of their business value

The main fact measures should include daily return, average daily return, beta, annual volatility, and optional derived measures such as rolling volatility, cumulative return, risk-adjusted return, and cluster membership history. This design is appropriate because the assignment's analysis is driven by the interaction among return, market sensitivity, and volatility, and because the chosen four-cluster segmentation turns raw market data into business-ready investor groups for targeting, monitoring, and rebalancing decisions. It also supports trend analysis, dashboarding and future predictive extensions.

C-2a. Review of Results

The derived stock clusters provide clear business value for investors because they convert a very large universe of S&P 500 stocks into a small number of interpretable risk-return groups that can be used for portfolio construction, rebalancing and investor targeting. In Task B, both the agglomerative and k-means analysis supported a four-cluster solution as the most useful balance between detail and interpretability. The selected clusters can be viewed as Defensive Low-Risk stocks, Stable Core holdings, Moderate Growth stocks, and Aggressive High Risk stocks. This structure is valuable because it gives investors meaningful portfolio categories rather than an unmanageable list of individual securities.

The business value of the clustering depends on the interaction of the three metrics. **Daily Return** represents the reward dimension, showing the average daily gain or loss of a stock. **Beta** measures the stock's sensitivity to general market movements and therefore captures systematic risk. **Annual Volatility** measures the overall instability of returns and captures the broader riskiness of the stock's behaviour. These metrics are related but not identical, which is why they are useful together. A stock may have a strong daily return but also a very high beta and volatility, meaning the return is achieved with substantial risk. Conversely, a lower-return stock with low beta and low volatility may be highly valuable as a stabiliser within a diversified portfolio.

For investors, the practical business value is improved decision-making. Conservative investors can favour defensive and core clusters, balanced investors can combine core and moderate-growth clusters, and more aggressive investors can selectively include high-risk stocks alongside stabilising holdings. This cluster-based view therefore supports clearer portfolio design, better risk communication, and more disciplined portfolio rebalancing.

C-2b. Python Visualisations

All Python code used to generate the following visualisations is available as a Google Colab notebook in the GitHub repo.

These visualisations summarise the business value of the derived stock clusters using the three key metrics: Daily Return, Beta, and Annual Volatility.

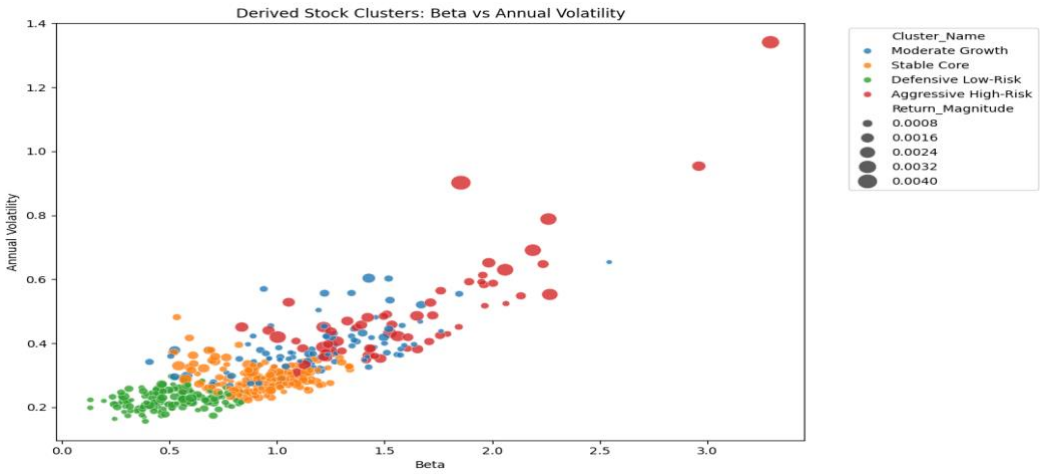


Figure 18. Derived Stock Clusters: Beta versus Annual Volatility

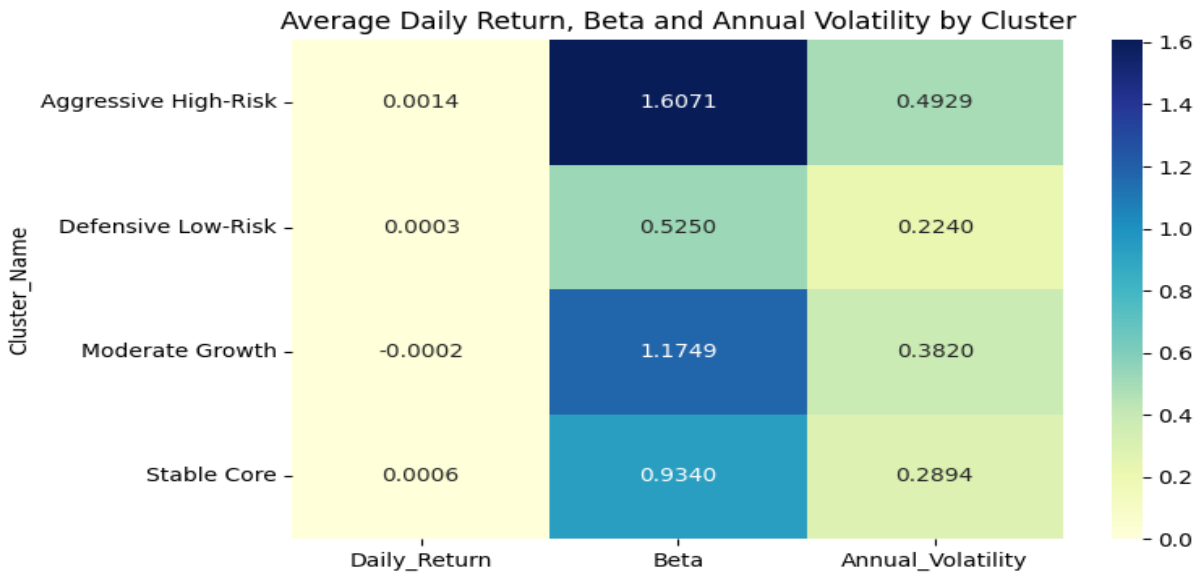


Figure 19. Average Daily Return, Beta and Annual Volatility by Cluster

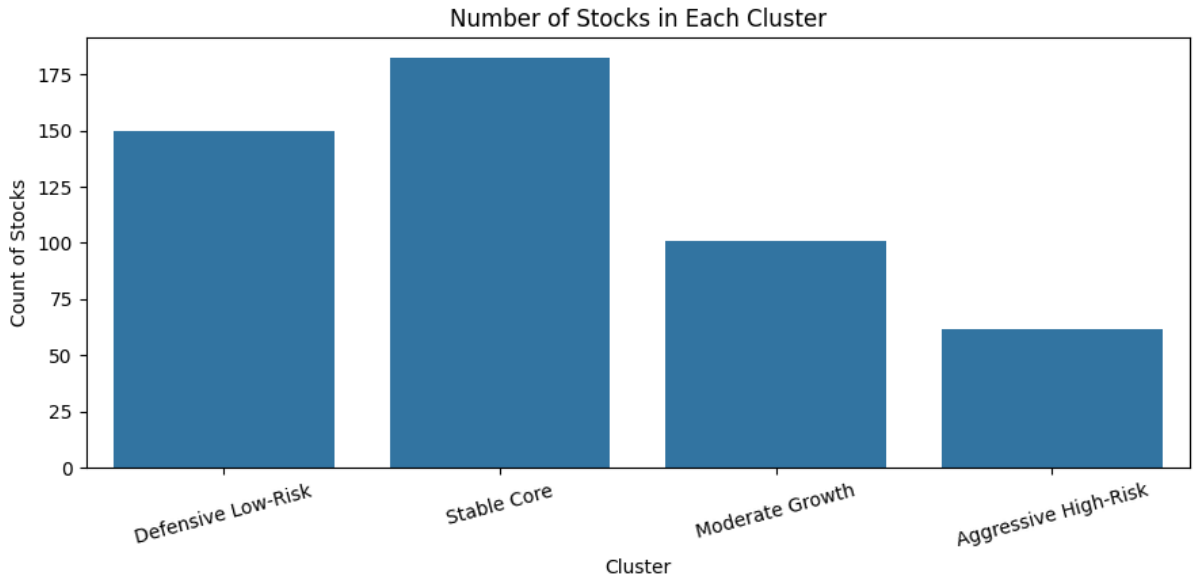


Figure 20. Number of Stocks in Each Derived Cluster

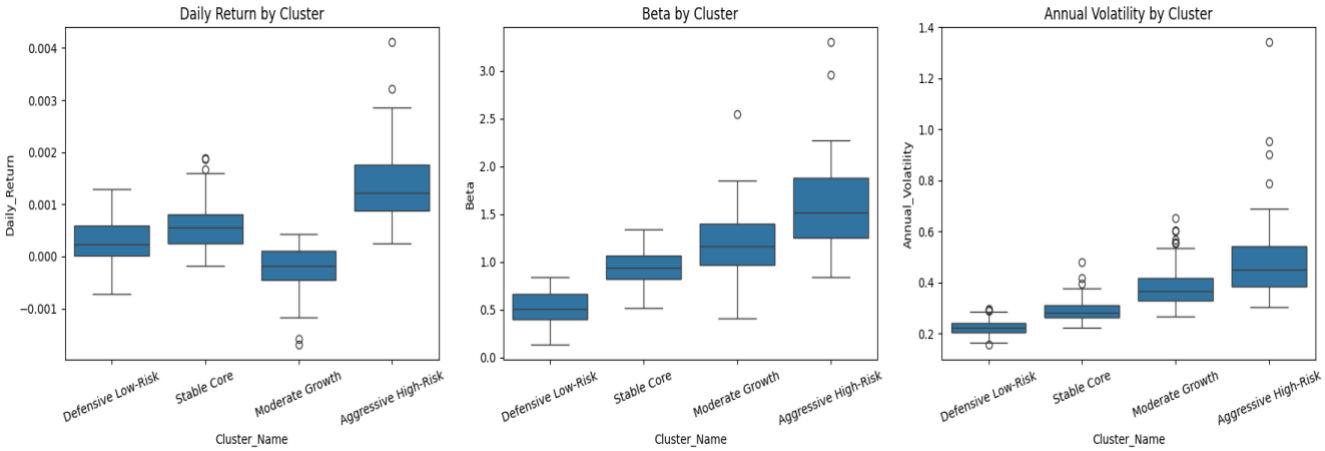


Figure 21. Distribution of Daily Return, Beta and Annual Volatility by Cluster

C-3. How CRM tools contribute to sustainability for Fintech

Customer relationship management (CRM) tools support sustainability in Fintech by focusing on long-term customer outcomes rather than short-term sales. CRM consolidate customer data, interaction histories, and service journeys, enabling firms to better understand needs, personalise support, and monitor risks effectively. While digital financial services expand access, lower costs, and improve transparency, poor customer data use can pose risks such as fraud and privacy breaches (World Bank, 2020; OECD, 2020). Thus, the paragraphs below will explore how the use of CRM tools helps avoid this and helps Fintech firms attain sustainability.

The first benefit of CRM tools is economic sustainability. Fintechs often grow rapidly, but high acquisition costs and inefficiencies hinder long-term viability. CRM tools aid by enhancing segmentation, communication, and customer retention. Personalisation tailored to individual risk tolerance and needs drives revenue and improves customer experience, supporting profitable growth (McKinsey, 2021).

Second, CRM tools promote operational sustainability by automating onboarding, complaint handling and supporting workflows, reducing costs and increasing scalability. As noted by the World Bank (2020), data analytics improve efficiency and service quality, enabling fintechs to allocate resources more effectively, intervene early when customers disengage, and refine products over time.

Third, CRM tools foster social and governance sustainability. Firms must ensure fairness, transparency, and trust, in line with the FCA 2026 guidelines, and avoid risks such as fraud and data breaches highlighted by the OECD (2020). An effective CRM strategy combines analytics, consent management, fair targeting, complaint monitoring, and customer support, helping fintech attain these goals while growing sustainably and building trust for a more resilient industry.

C-3 References

Financial Conduct Authority (2026). *About the Consumer Duty*. Financial Conduct Authority. Available from the Financial Conduct Authority website [Accessed 24 April 2026].

McKinsey & Company (2021). *The value of getting personalization right—or wrong—is multiplying*. McKinsey & Company. Available from the McKinsey & Company website [Accessed 24 April 2026].

Organisation for Economic Co-operation and Development (2020). *Financial Consumer Protection Policy Approaches in the Digital Age: Protecting consumers' assets, data and privacy*. OECD. Available from the OECD website [Accessed 24 April 2026].

World Bank (2020). *Digital Financial Services*. World Bank. Available from the World Bank website [Accessed 24 April 2026].